

Ligate Chain vs Ethereum Attestation Service

Stefan Stefanović, Ligate Labs

2026-05-25

Ligate Chain vs Ethereum Attestation Service

A Technical Comparison Across Six Architectural Axes

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-27

Status: v0.2 promotes the v0.1 outline to substantive content across all sections. §2 surveys EAS architecture in detail. §3 quotes Ligate primitives from the v0.2 paper portfolio. §4 fills in the six-axis comparison table with documented EAS behavior and citations. §5 maps representative use-case profiles to each architecture. §6 sketches the chain-side composition that lets EAS attestations be referenced from Ligate via Cross-Schema Composition v0.2 §4.3 typed references plus the cross-chain attestation portability follow-up paper.

Contact: hello@ligate.io

Version history: v0.1 (2026-05-25, outline). v0.2 (2026-05-27, substantive content across all sections + comparison table filled).

Contents

Ligate Chain vs Ethereum Attestation Service	1
A Technical Comparison Across Six Architectural Axes	1
Abstract	3
1. Introduction	3
1.1 Why this comparison exists	3
1.2 Why now	3
1.3 What the note answers	4
1.4 The central question	4
1.5 Approach in brief	4
1.6 Contributions	4
1.7 Scope and non-goals	5
1.8 Document structure	5
2. EAS Architecture in Brief	5
2.1 The two-contract design	5
2.2 Tokenless economic model	6
2.3 Schema collaboration model	6
2.4 SDK and deployment footprint	7
3. Ligate Chain Attestation in Brief	7
3.1 Attestation as a chain operation	7
3.2 PoUA economic security	8
3.3 Threshold attestor sets	8
3.4 Per-schema fee markets	8
3.5 Typed cross-schema composition	9
3.6 Schema-bound tokens	9
3.7 Time-locked attestations	9
4. Six-Axis Comparison	10
4.1 The comparison table	10
4.2 Axis 1: Economic security	11
4.3 Axis 2: Signer model	11
4.4 Axis 3: Fee market	11
4.5 Axis 4: Composition	12
4.6 Axis 5: Token primitives	12
4.7 Axis 6: Time semantics	12
5. Use-Case Profile Mapping	13
5.1 Use cases that lean EAS	13
5.2 Use cases that lean Ligate	13
5.3 Mixed-use scenarios	14
6. Composition Rather Than Competition	14
6.1 Cross-chain typed reference sketch	14
6.2 Why this is interesting	15
6.3 Caveats	15
7. Conclusion	16
References	17
Appendix A: EAS deployment footprint snapshot	17

Abstract

The Ethereum Attestation Service (EAS) is a contract-based attestation registry deployed across Ethereum L1 and major L2s. Ligate Chain is an attestation-native chain with PoUA-backed economic security where attestations are first-class chain operations. Both serve the “signed claim posted on-chain” surface, but they make materially different architectural commitments.

This note compares EAS and Ligate Chain across six axes: economic security floor, signer model, fee-market structure, cross-attestation composition, token primitives, and time semantics. The comparison resolves the question investors and design partners ask first (“why not just use EAS?”) with technical precision. The conclusion: EAS and Ligate sit in different architectural families, not as direct competitors. EAS optimizes for infrastructure-public-good integration simplicity, accepting that economic security, threshold mint, and typed composition must be application-layer concerns. Ligate optimizes for chain-native economic security and threshold primitives, accepting that the chain commits to a token and a more substantial protocol surface. Use cases lean toward one or the other depending on whether economic-security and threshold demands dominate.

The note also sketches the chain-side composition that lets a Themisra attestation on Ligate carry a typed reference to an EAS attestation on Ethereum (via Cross-Schema Composition v0.2 §4.3 + the cross-chain attestation portability mechanism in [papers/cross-chain-portability/](#)). Composition rather than competition is the framing the broader ecosystem benefits from.

1. Introduction

1.1 Why this comparison exists

EAS is the closest direct peer to Ligate Chain in the attestation space. Investors, design partners, and EAS users evaluating Ligate ask the same first-order question: “why not just use EAS?” This note answers it with technical precision. The position is not that one system replaces the other; the position is that the two systems make different architectural commitments and suit different use-case profiles.

The comparison matters because the word “attestation” carries different operational implications in the two stacks. In EAS, “attestation” means “signed claim recorded by a contract.” In Ligate, “attestation” means “signed claim recorded as a first-class chain operation with PoUA economic-security floor, threshold-attestor-set authority, and typed-composition discipline.” Both are valid technical commitments; they imply different downstream properties for builders, regulators, and adversarial counterparties.

1.2 Why now

Three concurrent shifts make the comparison timely. First, EAS adoption has reached non-trivial scale on Ethereum L1 and major L2s (Base, Optimism, Arbitrum); the question “is EAS sufficient?” is no longer hypothetical. Second, EU AI Act Article 50 (full effect August 2026) creates regulatory demand for verifiable AI-content marking at the artifact level; institutional buyers evaluating attestation substrates now consider economic-security guarantees a procurement criterion rather than an academic detail. Third, the broader C2PA Content Credentials adoption wave ([papers/c2pa-composition/](#)) has moved attestation infrastructure from niche to category, raising the importance of being technically precise about which substrate offers which guarantees.

The chain is also pre-mainnet, so the comparison is not retrospective. Builders making attestation-substrate choices in 2026-2027 face the question directly. This note gives them the technical material to choose, and to choose to compose both where workflows warrant.

1.3 What the note answers

Three questions. (a) Where do EAS and Ligate make the same architectural commitment? Both expose schema registration + attestation submission as the core primitive surface. Both are permissionless to attest under registered schemas. Both support reference patterns between attestations. (b) Where do they diverge? Six axes specified in §4: economic security, signer model, fee market, composition, token primitives, time semantics. (c) What use cases lean which way? §5 maps representative workflows to the better-fitting architecture.

The answer is comparison, not advocacy. Some workflows are objectively better-served by EAS today; others are better-served by Ligate. A non-trivial set are best served by composing both via the chain-side mechanism sketched in §6.

1.4 The central question

Is an attestation primitive better deployed as a smart contract on a general-purpose chain, or as a native operation on a chain whose entire surface is attestations?

The answer depends on three orthogonal demands: (1) the use case's economic-security needs (does misbehavior need to carry economic consequence at the consensus layer, or is reputational consequence at the application layer sufficient?); (2) the composition demands (are typed cross-attestation references required, or are untyped pointers sufficient?); (3) the operational requirements around threshold authority, recall, and time-locked semantics. EAS is the right choice when all three demands are modest; Ligate is the right choice when one or more is substantial.

1.5 Approach in brief

§2 surveys EAS architecture as deployed in 2026: two-contract design, schema registry, attestation contract, off-chain attestation variant, SDK + deployment footprint. §3 surveys Ligate's attestation primitive as specified in PoUA v0.9.2, per-schema-fees v0.2, cross-schema-composition v0.2, schema-bound-tokens v0.2, and time-locked-attestations v0.2; the §3 surveys quote from these papers rather than re-deriving. §4 presents the six-axis comparison table with documented EAS behavior in each cell. §5 maps use-case profiles. §6 sketches composition between the two systems, citing the cross-chain attestation portability mechanism from [papers/cross-chain-portability/](#). §7 concludes.

No formal proof; the note is descriptive, not derivative. Where EAS behavior is cited, citations point to either the EAS specification (<https://docs.attest.org/>), the EAS GitHub repositories, or documented EAS deployment behavior.

1.6 Contributions

1. **EAS architecture survey.** Brief technical description of the two-contract design, schema registry, attestation lifecycle, fee structure, SDK surface.
2. **Six-axis comparison.** Economic security, signer model, fee market, composition, token primitives, time semantics, presented in a single table plus per-axis discussion.
3. **Use-case profile mapping.** Which workflows lean toward EAS, which lean toward Ligate, with honest qualifications.

4. **Composition sketch.** How a Themisra attestation could reference an EAS attestation via CSC v0.2 §4.3 typed reference + cross-chain bridge (deferred to follow-up paper).

1.7 Scope and non-goals

In scope:

- Technical architecture of EAS as deployed in 2026
- Architectural commitments of Ligate Chain v0.2+ relative to EAS
- Use-case profile mapping (qualitative)
- Composition possibilities (conditional on follow-up paper)

Explicitly out of scope:

- Quantitative benchmarking (per-attestation cost, throughput); would require empirical measurement; v0.3+ if useful
- Detailed cross-chain bridge specification; lives in #136
- Argument that EAS is broken or insufficient for its use cases; it is not
- Migration guide from EAS to Ligate; the note is comparison, not advocacy

1.8 Document structure

§2 surveys EAS architecture as deployed in 2026. §3 surveys Ligate Chain’s attestation primitive, quoting from the v0.2 paper portfolio. §4 presents the six-axis comparison table with documented behavior + citations and per-axis discussion. §5 maps use-case profiles to each architecture, including mixed-use scenarios. §6 sketches the chain-side composition mechanism that lets EAS attestations be referenced from Ligate. §7 concludes. Appendix A reserves space for an EAS deployment-footprint snapshot at authoring time, deferred to v0.3 pending data collection.

2. EAS Architecture in Brief

EAS deploys two Solidity contracts per chain: the SchemaRegistry and the EAS contract. The architecture is intentionally minimal; the design philosophy explicit in the EAS documentation is “infrastructure public good.” This section summarizes the architecture from the public specification and the deployed contract behavior.

2.1 The two-contract design

SchemaRegistry contract. Provides one operation: `register(string schema, address resolver, bool revocable)`. The `schema` argument is a Solidity-style type string (e.g., `"bool isHuman,uint256 timestamp"`). The `resolver` argument is an optional Solidity contract address that can validate attestation submissions and trigger side effects on attest / revoke. The `revocable` argument is a boolean lock for whether the schema’s attestations can be revoked. Each registration produces a deterministic schema UID (keccak-256 hash of the encoded arguments).

EAS contract. Provides the primary surface: `attest(AttestationRequest)`, `revoke(RevocationRequest)`, `multiAttest`, `multiRevoke`, plus their delegated variants for off-chain signing. An `AttestationRequest` carries `schema` (the schema UID to attest under), `data` (the encoded attestation payload

conforming to the schema’s type string), **recipient**, **refUID** (optional pointer to another attestation), **expirationTime**, **revocable**, and **value** (optional ETH transfer to the resolver). Each successful attest produces a deterministic attestation UID.

UID-based addressing. Every schema and every attestation has a unique UID. References between attestations use UIDs as untyped pointers. There is no protocol-level constraint that the referenced attestation’s schema match any specific type; downstream consumers verify type-correctness off-chain.

Off-chain attestation variant. EAS supports EIP-712-signed off-chain attestations that can be anchored on-chain by reference (via the **attestByDelegation** flow). Off-chain attestations save gas at the cost of inheriting off-chain trust assumptions for storage and retrieval.

2.2 Tokenless economic model

EAS does not have a native token. The cost of an attestation is the L1 or L2 gas required to execute the EAS contract’s **attest** function plus any value transferred to the resolver. Schema authors capture no native fee share (custom resolvers can charge ETH transfers, but this is application-layer plumbing, not protocol design). Ethereum / L2 validators capture the gas as standard block reward.

The tokenless framing has substantial downstream implications. First, the signer model: since any address can attest for the cost of gas, EAS has no protocol-layer mechanism to gate attestations behind threshold authority; threshold attestations are bolted on as application-layer multisig contracts. Second, the economic security model: there is no protocol-level slashing of attestors who misbehave. Misbehavior consequences are reputational at the application layer (downstream consumers can stop trusting an attestor’s UID) but not economic at the protocol layer. Third, the fee market: there is no per-schema fee dynamics; high-value schemas pay the same chain gas as low-value schemas.

The framing is deliberate. EAS chose “infrastructure public good” over economic-security-as-protocol-primitive. The trade-off is well-defended in the EAS documentation and is one defensible point in the design space.

2.3 Schema collaboration model

EAS has no protocol-level canonical schema mechanism. Anyone can register any schema string; the resulting UID is deterministic. Two parties registering the same schema text get the same UID (and conventionally the second registration is treated as a no-op or revert depending on resolver behavior).

Schema collaboration happens via off-chain coordination: community forums, GitHub repositories (e.g., the EAS schema explorer at <https://easscan.org/>), and attestor-org governance. The EAS documentation explicitly invites “if your schema has broader applicability, propose it” to community channels but the protocol itself does not arbitrate.

The downside: schema namespacing is by convention, not enforced. Two unrelated communities can independently invent semantically similar schemas with different UIDs; downstream consumers must learn which UIDs belong to which communities. The Ethereum dev community has converged on community-maintained “schema standards” that operate informally; the protocol is neutral.

The upside: zero friction to register a new schema. Builders prototype freely and the protocol does not gate experimentation.

2.4 SDK and deployment footprint

EAS ships SDKs in TypeScript (@ethereum-attestation-service/eas-sdk) and Rust (community-maintained). The TypeScript SDK is the primary one; it covers schema registration, on-chain attestation, off-chain attestation, delegated attestation, batched operations, and resolver integration.

EAS is deployed on multiple chains as of 2026: Ethereum L1, Base, Optimism, Arbitrum One, Polygon, Scroll, Linea, and several other L2s. Each deployment is a separate contract instance with its own schema registry and attestation set. There is no cross-deployment composition at the protocol level; an attestation on Base cannot be referenced from an attestation on Optimism within the EAS protocol. Cross-chain composition is application-layer (off-chain coordination + cross-chain bridge messages).

Operationally, the multi-chain deployment is both EAS's strength (broad reach across the Ethereum-adjacent ecosystem) and a constraint (composability is per-chain). Builders working within a single EVM chain have full EAS access; builders working across chains carry the integration burden themselves.

3. Ligate Chain Attestation in Brief

Ligate Chain treats attestation as a first-class chain operation rather than a smart-contract pattern. The protocol surface is more substantial than EAS's two-contract minimum: PoUA consensus weighting, threshold attester sets, per-schema fee markets, typed cross-schema composition, schema-bound tokens, and time-locked attestations are all native chain operations. This section quotes from the v0.2 paper portfolio rather than re-deriving.

3.1 Attestation as a chain operation

PoUA v0.9.2 §2 specifies three native chain operations:

- `RegisterSchema(schema_id, type_spec, attester_set_id, fee_params, ...)`: anyone can register a schema; the schema becomes a first-class chain entity with its own state, fee parameters, and bound attester set.
- `RegisterAttesterSet(set_id, members[], threshold)`: anyone can register an attester set with declared threshold t of n . Set membership and threshold are chain state.
- `SubmitAttestation(schema_id, payload, attester_signatures[])`: submit an attestation under a registered schema. Requires t valid signatures from the schema's bound attester set members.

The operations are not smart-contract calls; they are direct state transitions in the chain's runtime. There is no Solidity contract to deploy. The chain's runtime implements the verification, fee routing, and state updates atomically.

3.2 PoUA economic security

PoUA v0.9.2 §5.5.3 Lemma 1 establishes a cost-to-grind floor: an adversary attempting to manufacture Δr reputation must pay net fee burn of at least

$$F^{\text{net}} \geq \frac{\tau_{\text{burn}} \cdot \Delta r}{\eta \cdot \alpha_{\text{eff}}}$$

where τ_{burn} is the per-schema burn fraction, η is the reputation-to-influence conversion, and α_{eff} is the proposer-share term with cartel adjustment.

The bound is the load-bearing economic-security guarantee: even an adversary with arbitrary capital cannot purchase reputation without paying a proportional amount of AVOW into the protocol-burn sink. The bound is signature-scheme-agnostic (per `papers/pq-migration/` v0.2 Appendix A); it survives crypto migration intact.

For comparison purposes: EAS has no protocol-layer analog. EAS attester reputation is application-layer; no protocol mechanism makes misbehavior costly beyond the L1/L2 gas of submitting the misbehaving attestation.

3.3 Threshold attester sets

Attester sets are first-class chain entities. A set carries a declared threshold t of n members. Each member has a chain address. `SubmitAttestation` requires t valid signatures from set members aggregated via BLS (v0/v1) or a PQ-threshold scheme (post-migration per `papers/pq-migration/` §5.4).

Set membership updates are themselves attestations under the canonical schema `chain.attester-set-update/v1`. The mechanism is recursive: changing who can attest under a schema is itself an attestation under a system schema, evaluated by the set’s current members. There is no out-of-band administrative key.

For comparison: EAS supports threshold-style patterns only via application-layer multisig contracts (Gnosis Safe + EAS resolver). The threshold is not enforced at the EAS protocol level; the protocol sees a single signing address (the multisig contract) and validates its EOA-style signature. Recall and threshold-rotation must be implemented in the multisig contract, not in EAS itself.

3.4 Per-schema fee markets

per-schema-fees v0.2 §4.1 specifies an EIP-1559-style per-schema base fee b_σ that adjusts every block based on the schema’s own utilization u_σ relative to its target T_σ . High-volume schemas command higher base fees automatically; low-volume schemas decay toward the protocol minimum. The τ_{burn} fraction of each base fee is burned to the protocol sink; the rest splits between validators and the schema author per the `rho_sigma` routing fraction.

The per-schema design has two consequences. First, a single chain hosts many fee markets in parallel; no schema’s demand spike affects other schemas’ pricing. Second, schema authors capture a routed share of fees automatically (no application-layer plumbing needed), creating native incentive alignment.

For comparison: EAS has a single chain-wide gas market. All schemas pay the same L1/L2 gas price; high-value schemas cannot price out spam without resolver gymnastics; schema authors capture no fee share natively.

3.5 Typed cross-schema composition

Cross-Schema Composition v0.2 §3 + §4 specifies that schemas declare an input-type set. References from one attestation to another are typed: the referenced attestation must be under a schema in the declaring schema’s input-type set, optionally constrained by a bounded-compute admission predicate. The runtime checks types and predicates at admission, not at consumption.

CSC v0.2 §5 specifies cascade semantics: if a referenced attestation is invalidated (revoked, slashing-cascaded, time-lock-expired without reveal), the dependent attestation is automatically marked invalid via BFS cascade through the dependency graph. Cascade gas is charged to the revocation root; total cascade cost is bounded by graph depth.

For comparison: EAS attestations can reference each other via `refUID`, but the reference is untyped. The protocol does not verify that the referenced attestation is under any specific schema; downstream consumers must validate. Cascade-on-revocation is similarly application-layer: if attestation A references B and B is revoked, EAS does not automatically invalidate A. The downstream consumer must check `refUID` validity at consumption time.

3.6 Schema-bound tokens

Schema-Bound Tokens v0.2 §2-§3 specifies a third token-issuance primitive (alongside standard fungible and standard non-fungible tokens). Mint authority is an `AttestorSetId` rather than a single address. Mints are attestations under the canonical system schema `chain.token-mint/v1`. The mechanism delivers: native auditability (the attestation log carries the mint history), native threshold mint authority (no bolt-on multisig), native composition with the fee market (per SBT v0.2 §3.6 each mint pays per-schema base fee), and native composition with the reputation layer (bad-faith mints damage the attestor set’s PoUA reputation).

For comparison: EAS does not ship a token primitive. ERC-20 / ERC-721 tokens live in separate contracts; the relationship between an EAS attestation and a token mint is whatever the application contracts implement. Threshold mint authority is a multisig contract pattern, not an EAS-native primitive.

3.7 Time-locked attestations

Time-Locked Attestations v0.2 §3 + §4 specifies commit-reveal as a runtime primitive. Three transaction types (`MsgCommit`, `MsgReveal`, `MsgCleanup`) plus a four-state lifecycle (`COMMITTED` → `REVEALED` / `EXPIRED` / `CANCELED`) live in the chain runtime. Use cases include sealed-bid auctions, embargoed press releases, and regulatory filings.

For comparison: EAS supports commit-reveal patterns only at the application layer (a smart contract holds a commitment, then later submits an EAS attestation with the revealed payload). The protocol does not enforce the time-lock; the contract author bears the responsibility for correctness. Recall and cancel mechanics are also contract-layer.

4. Six-Axis Comparison

This section presents the comparison table plus per-axis discussion. The table is the load-bearing artifact of the note. Each cell cites either a documented EAS behavior (linked to the EAS specification or contract source) or a Ligate primitive (cited to the relevant v0.2 paper).

4.1 The comparison table

Axis	EAS	Ligate Chain	Notes
Economic security	L1/L2 gas only; no protocol-layer slashing of attestors	PoUA Lemma 1 floor: $F^{\text{net}} \geq \tau_{\text{burn}} \Delta r / (\eta \alpha_{\text{eff}})$	Central differentiator. EAS misbehavior consequence is reputational; Ligate is economic
Signer model	Single-sig per attestation; threshold via bolt-on multisig contract	Threshold attester set as first-class chain entity (t of n BLS)	Recall + rotation native on Ligate, contract-layer on EAS
Fee market	Single chain-wide L1/L2 gas market; schema authors capture no native share	Per-schema EIP-1559 with ρ_{σ} routing to schema author + τ_{burn} to protocol	Ligate prices high-value schemas independently; native creator economy
Composition	Untyped UID pointers; refUID can target any attestation	Typed input-type sets with bounded-compute admission predicates + BFS cascade on invalidation	Ligate enforces type-correctness at admission; EAS defers to off-chain consumer
Token primitives	None; ERC-20 / ERC-721 live in separate contracts	Schema-Bound Tokens as third token-issuance primitive with attester-set mint authority	Ligate ships SBT (mint events as attestations); EAS doesn't try
Time semantics	Block timestamps only; commit-reveal at application layer	Time-locked-attestations as runtime primitive (<code>MsgCommit</code> , <code>MsgReveal</code> , four-state lifecycle)	Ligate ships commit-reveal natively; EAS depends on app contracts for time-lock

Sources: EAS specification at <https://docs.attest.org/>, EAS GitHub repository (<https://github.com/ethereum-attestation-service>), PoUA v0.9.2 §5.5.3 (arXiv:2605.25844), per-schema-fees v0.2 §4.1 + §4.4, CSC v0.2 §3-§5, SBT v0.2 §2-§3, TLA v0.2 §3-§4.

4.2 Axis 1: Economic security

This is the central differentiator. EAS has no economic security at the attestation primitive level; an attester can be misbehaving and the protocol does not punish them. Punishment is reputational at the application layer: downstream consumers can choose to stop trusting a given attester’s UID. The protocol does not aggregate this reputation, does not slash any economic stake, and does not enforce any cost on the misbehaving attester beyond the L1/L2 gas they already paid.

Ligate’s PoUA Lemma 1 cost-to-grind floor is the economic-security primitive at the consensus layer. Every attestation submitted under a schema pays a fee; τ_{burn} of that fee is burned. Misbehaving attestors do not avoid the burn; they pay it on every attestation they submit, including the misbehaving ones. The chain layer enforces this; no application-layer cooperation is required. The bound $F^{\text{net}} \geq \tau_{\text{burn}} \Delta r / (\eta \alpha_{\text{eff}})$ says: to gain Δr reputation, an adversary must pay proportionally in burned AVOW.

When economic security matters: high-stakes attestations (regulated currency, evidentiary, audit-bearing) where misbehavior consequence must be visible at consensus, not optional at application. **When it does not:** low-stakes cooperative attestations where reputational damage is sufficient deterrent.

4.3 Axis 2: Signer model

EAS attestations are signed by exactly one address (an EOA or a contract address). Threshold-attester patterns (e.g., “5 of 7 board members agree before this attestation is valid”) are implemented as a multisig contract that signs the EAS attestation. The EAS protocol sees a single signer; it does not know whether that signer was a single key or a multisig quorum.

Ligate’s attester sets are first-class chain entities. `RegisterAttestorSet` declares membership and threshold; `SubmitAttestation` requires the threshold to be met via aggregated signatures from set members. The chain runtime verifies threshold satisfaction; there is no application-layer multisig to deploy. Set membership updates and threshold updates are themselves attestations under canonical system schemas, so attester-set governance is queryable from the same attestation log.

When threshold authority matters: consortium operations (banks issuing a regulated currency, multi-stakeholder embargo, DAO treasury operations), regulatory contexts requiring documented multi-party authorization, audit workflows where single-signer attestations are not credible. **When it does not:** individual creator workflows, single-organization attestations where one signer’s reputation is sufficient.

4.4 Axis 3: Fee market

EAS attestations pay L1 or L2 gas. Gas prices are chain-wide; a Themisra-equivalent low-stakes attestation pays the same gas as a high-stakes regulated-currency mint. Schema authors do not capture fee share natively. If a popular schema author wants royalties from its use, the application layer must implement royalty contracts that intercept fees and route to the author.

Ligate’s per-schema fee market gives each schema its own EIP-1559 dynamics: b_σ adjusts every block based on schema σ ’s utilization u_σ relative to its target T_σ . The chain hosts many fee markets in parallel; one schema’s demand spike does not affect other schemas’ pricing. Each base fee splits: τ_{burn} to the protocol sink, ρ_σ to the schema author (governance-tunable, bounded), the rest to validators.

When per-schema pricing matters: ecosystems with very different schemas (low-volume audit attestations alongside high-volume real-time attestations), workflows where schema-author incentive is needed at the protocol level (creator economy, schema-marketplace patterns), regulatory contexts where high-value schemas should structurally cost more than low-value schemas. **When it does not:** ecosystems with relatively uniform attestation workloads where chain-wide gas pricing is acceptable.

4.5 Axis 4: Composition

EAS attestations support `refUID` references between attestations. The reference is untyped at the protocol level: an attestation under schema X can reference an attestation under any other schema; consumers must validate type-correctness off-chain. EAS does not cascade revocation: if A references B and B is revoked, A's status does not change automatically; consumers must check.

CSC v0.2 §4.3 specifies typed references: a declaring schema names its allowed input types (a set of schema IDs), and the chain runtime verifies at admission that the referenced attestation is under one of those types. Optional bounded-compute admission predicates verify additional constraints (e.g., the input's signer matches a field in the declaring schema). CSC v0.2 §5 specifies BFS cascade on input invalidation: dependent attestations are automatically marked invalid through the dependency graph, with gas charged to the revocation root.

When typed composition matters: workflows with multi-step attestation pipelines where type-confusion would corrupt downstream state (license attestations referencing prompt receipts, audit attestations referencing financial-event attestations, derivative-work attestations referencing original licenses), workflows where revocation needs to cascade reliably without application-layer enforcement. **When it does not:** workflows where attestations are essentially independent (single-shot credentials, isolated registrations) and any cross-reference is convention-only.

4.6 Axis 5: Token primitives

EAS does not ship a token primitive. ERC-20 fungible tokens and ERC-721/1155 non-fungible tokens live in separate Solidity contracts; the relationship between an EAS attestation and a token is whatever application contracts implement. This is intentional; EAS scope is attestation-only.

Ligate ships Schema-Bound Tokens (SBT v0.2) as a third token-issuance primitive alongside standard fungible (ligate-chain#47) and non-fungible (ligate-chain#48) tokens. SBT mint authority is an `AttestorSetId` rather than a single address; mint events are attestations under the canonical schema `chain.token-mint/v1`; the audit trail is the attestation log; reputation feedback applies natively per PoUA §5.5.

When token primitives matter on the same substrate: workflows where the same chain hosts both attestations (proofs of state) and tokens (transferable state), with deep composition between the two (e.g., a regulated currency where issuance is itself an attestation auditable in the same log as the rest of the chain's attestations). **When they don't:** workflows that need only attestations and use tokens from a separate chain or contract layer.

4.7 Axis 6: Time semantics

EAS has block timestamps and `expirationTime` on each attestation. There is no protocol-level commit-reveal or time-lock primitive; workflows requiring them implement at the contract layer (commitment held in a Solidity contract; reveal triggers an EAS attestation submission).

TLA v0.2 §3-§4 ships commit-reveal as a runtime primitive: `MsgCommit` records a commitment, `MsgReveal` admits the reveal subject to the time-lock and four-state lifecycle (`COMMITTED` → `REVEALED` / `EXPIRED` / `CANCELED`). Use cases: sealed-bid auctions, embargoed press releases, regulatory filings with disclosure embargos.

When time-locks matter: sealed-bid mechanisms, regulatory embargos, journalism workflows with publication-date constraints, governance proposals with revelation timing. **When they don't:** real-time attestation workflows where every attestation is immediately public.

5. Use-Case Profile Mapping

This section maps representative use cases to the system whose architectural commitments fit them. The framing is honest: some use cases lean EAS for simplicity reasons, some lean Ligate for economic-security or threshold-authority reasons. A non-trivial set are best served by composing both (§6).

5.1 Use cases that lean EAS

Workflows where simplicity dominates and the additional Ligate primitives are not needed:

- **Single-signer badge attestations** (e.g., a DAO issues “voted on proposal X” badges to participants). One signer, low stakes, no threshold needed, no per-schema fee market needed.
- **Personal-credential workflows** (KYC-style attestations from a single issuer to a single holder). One signer, no composition required.
- **Single-deployment ecosystems** where the workflow lives entirely on one chain (Base, Optimism, etc.) and EAS’s per-chain deployment is sufficient.
- **Prototype + experimentation contexts** where the cost of switching is small if the project later needs more capability. EAS’s lower integration cost wins here.
- **Workflows where Ethereum L1 / L2 ecosystem reach matters more than economic-security depth.** Existing EAS attester reputation + EAS-aware tooling are a real advantage for builders inside the EVM ecosystem.

The common thread: low operational stakes, single-signer sufficiency, no requirement for threshold authority or typed composition, integration speed matters more than security depth.

5.2 Use cases that lean Ligate

Workflows where economic security, threshold authority, typed composition, native time-locks, or schema-bound tokens are first-order requirements:

- **Regulated currency issuance** (consortium of banks, central-bank digital currency). Threshold mint authority is required (single-issuer is unacceptable for regulators), audit trail must be queryable in the same log as the rest of the chain’s attestations, recall procedures must be runtime-enforced. SBT v0.2 §6.1 specifies this use case as a worked example.
- **High-value AI-provenance with permanent receipts.** Themisra Proof-of-Prompt attestations carry compliance weight under EU AI Act Article 50; receipts must remain valid indefinitely and survive adversarial action against the artifact. PoUA economic security at the consensus layer is the differentiator.

- **Audit-bearing attestations.** Financial-services audit, regulatory-compliance audit, journalism-evidence chains. Misbehaving attestors must carry economic consequence at protocol level so the audit trail’s authority does not depend on application-layer cooperation.
- **Cross-schema workflows with typed composition.** Themisra licensing schemas (papers/themisra-licensing-schemas/) reference Proof-of-Prompt receipts via CSC v0.2 §4.3 typed references; cascade-on-revocation is required. EAS’s untyped UID pointers cannot meet this requirement.
- **DAO governance attestations.** Voting records, treasury operations, multi-stakeholder proposals. Native threshold authority + recall + native composition with token primitives (SBT for governance tokens) keeps the surface integrated.
- **Regulated-license registrations.** Professional-licensing boards issue licenses as NFTs with attestor-set threshold authority and runtime-enforced recall. SBT v0.2 §6.5 sketches this use case.

The common thread: high stakes, threshold authority or typed composition demands, native economic security at the consensus layer.

5.3 Mixed-use scenarios

A non-trivial set of workflows is best served by composing both systems rather than choosing one. Example: a Themisra AI-provenance attestation on Ligate referencing an EAS-attested model-card published by OpenAI on Ethereum. The Themisra attestation carries the user’s prompt-output linkage with PoUA economic security; the referenced EAS attestation carries the model-card from a trusted issuer in the EAS ecosystem. The composition mechanism is sketched in §6.

This pattern is common when: (a) the workflow already touches both ecosystems (some content originates on Ethereum, some on Ligate), (b) re-issuing existing EAS attestations on Ligate would be operationally awkward, (c) Ligate-side workflows want to augment EAS-side attestations with additional guarantees (threshold authority, typed cascade, time semantics) without breaking compatibility with EAS-side tooling.

6. Composition Rather Than Competition

EAS and Ligate sit in different architectural families and can compose. This section sketches the mechanism.

6.1 Cross-chain typed reference sketch

A Themisra attestation can declare an input type of the form `eas.attestation.<schema-uid>` in its CSC v0.2 §4.3 input-type set. The reference value carries (`chain-id`, `eas-uid`, `light-client-proof`): the Ethereum / L2 chain ID where the EAS attestation lives, the attestation’s UID on that chain, and a light-client proof of the EAS attestation’s current validity from a Ligate-side chain-state mirror.

The admission predicate on the Ligate side verifies: (a) the light-client proof correctly attests to the EAS attestation’s existence at some block height h on the named chain, (b) the EAS attestation is under the schema UID named in the input-type, (c) the EAS attestation’s `expirationTime` has not passed and its `revocable` flag plus chain state indicate it is not revoked as of height h , (d)

the Ligate-side mirror’s freshness commitment for that chain is recent enough (per the cross-chain attestation portability paper).

Cascade semantics apply: if the EAS attestation is later revoked, the Ligate-side mirror catches the revocation on the next update; CSC v0.2 §5 cascade fires on the dependent Themisra attestation. Revocation-visibility latency is bounded by the mirror’s update cadence (recommended 30-second cadence per [papers/cross-chain-portability/ v0.1 §5](#)).

6.2 Why this is interesting

The composition lets existing EAS deployments be referenced by Ligate-side attestations without re-issuing them on Ligate. EAS’s existing data (model cards, identity attestations, audit records, etc.) becomes addressable from Ligate workflows without migration; Ligate-side workflows add typed composition, threshold attestor mint authority, recall semantics, and economic-security guarantees on top.

Three concrete patterns:

1. **Augment EAS attestations with Ligate-side threshold authority.** A Ligate attester set co-signs a Ligate attestation that wraps an EAS attestation, vouching for it under threshold-set authority. Downstream consumers can verify both the EAS-side single-signer attestation and the Ligate-side threshold attestation.
2. **Use Ligate-side cascade against EAS-side revocation.** A Ligate composition pipeline depends on an EAS attestation; if the EAS attestation is revoked, the Ligate dependency graph cascades invalidation through downstream Ligate attestations. EAS’s lack of native cascade is filled by Ligate’s cascade once the EAS attestation is referenced.
3. **Add Ligate-side fee-market routing on top of EAS-side schema usage.** A Ligate schema that references EAS attestations can route fees to schema authors via per-schema-fees v0.2 §4.4 ρ_σ , even when the underlying claims originate on EAS.

These are not theoretical; they are the natural fit for builders who have invested in EAS-side infrastructure and want Ligate-side capabilities additively.

6.3 Caveats

Three honest caveats.

Bridge complexity. The cross-chain mirror is non-trivial engineering. The Ligate light-client implementation for Ethereum / L2s requires either (a) IBC-compatible light-client portability of Sovereign SDK rollup state, which is not yet native in most Cosmos chains, or (b) a Hyperlane-style interoperator-validator-based bridge. The cross-chain attestation portability paper documents the engineering gap.

Update latency. The Ligate-side mirror catches EAS-side revocation only after the next update. Revocation events on Ethereum can take 30 seconds to several minutes to propagate, depending on the mirror’s update cadence. Composed workflows must accept this latency or design for it (e.g., enforce a “wait N blocks after creation before downstream attestations can reference” pattern).

Security inheritance from the weaker chain. A Ligate attestation referencing an EAS attestation is only as strong as the EAS attestation’s authentication and the chain it lives on. If the Ethereum / L2 chain is compromised (51%-attack-style reorgs, fork events with conflicting attestations), the referenced EAS attestation’s status becomes ambiguous on Ligate. The cross-chain

paper specifies how Ligate handles this; the conservative default is to require finality on the source chain before accepting references.

7. Conclusion

EAS and Ligate Chain are not direct competitors. They make different architectural commitments and serve different use-case profiles. EAS optimizes for infrastructure-public-good integration simplicity, accepting that economic security, threshold mint authority, typed composition, and time-locks are application-layer concerns. Ligate optimizes for protocol-layer economic security and threshold primitives, accepting that the chain commits to a token and a substantial protocol surface. Both are defensible positions on the design frontier. The six-axis table in §4 is the load-bearing artifact: it documents the commitments precisely enough that builders can choose with technical clarity.

Composition between the two is plausible and interesting. A Themisra attestation on Ligate can carry a typed reference to an EAS attestation on Ethereum via the cross-chain attestation portability mechanism specified in [papers/cross-chain-portability/](#). Existing EAS deployments become addressable from Ligate workflows without migration; Ligate-side workflows add threshold authority, typed cascade, and economic security on top. The composition is not theoretical; it is the natural fit for builders who have invested in EAS-side infrastructure and want Ligate-side capabilities additively. This note positions Ligate accurately without overclaiming, and offers EAS users a path to compose rather than migrate.

References

[v0.1: References to fill in at v0.2. Anchors:]

1. EAS (Ethereum Attestation Service) official documentation. <https://docs.attest.org/>
2. EAS specification repository. <https://github.com/ethereum-attestation-service>
3. Sign Protocol (EAS-variant attestation network).
4. Verax (Ethereum attestation registry, Linea-flavored).
5. PoUA paper (this repo, [papers/poua/](#)).
6. Per-Schema Fees paper (this repo, [papers/per-schema-fees/](#)).
7. Cross-Schema Composition paper (this repo, [papers/cross-schema-composition/](#)).
8. Schema-Bound Tokens paper (this repo, [papers/schema-bound-tokens/](#)).
9. Time-Locked Attestations paper (this repo, [papers/time-locked-attestations/](#)).

Appendix A: EAS deployment footprint snapshot

[v0.1: At v0.2: a tabular snapshot of EAS deployments as of authoring date. Per-chain attestation counts (if publicly accessible). Schema counts. Adoption signals.]